AI EVALUATION & SECURITY FOR ENGINEERS

Shipping Faster with Confidence

> Practical performance and security evaluation for AI systems built on commercial models

The Core Thesis

1. We use off-the-shelf commercial AI models.
2. Our job isn't to evaluate training data or benchmark model internals.
3. Our job is to **evaluate the systems we build around them** (before deployment and in production)
4. so we can ship faster with less rework, fewer incidents, and more confidence.

CHAPTER 1 THE PROBLEM

Why AI Projects Fail Between Demo and Production

🧪 Demo World

- Clean prompts, clean outputs
- Curated examples
- No integrations
- No adversarial users

🌍 Production Reality

- Messy, ambiguous inputs
- Complex business workflows
- Crazy number of edge cases
- Potential adversarial users (for public facing apps)

ⓘ Demos succeed on clean examples. Production fails on messy reality. It's dangerously easy to feel 95% done when you're barely halfway there.

Why this is especially important for us

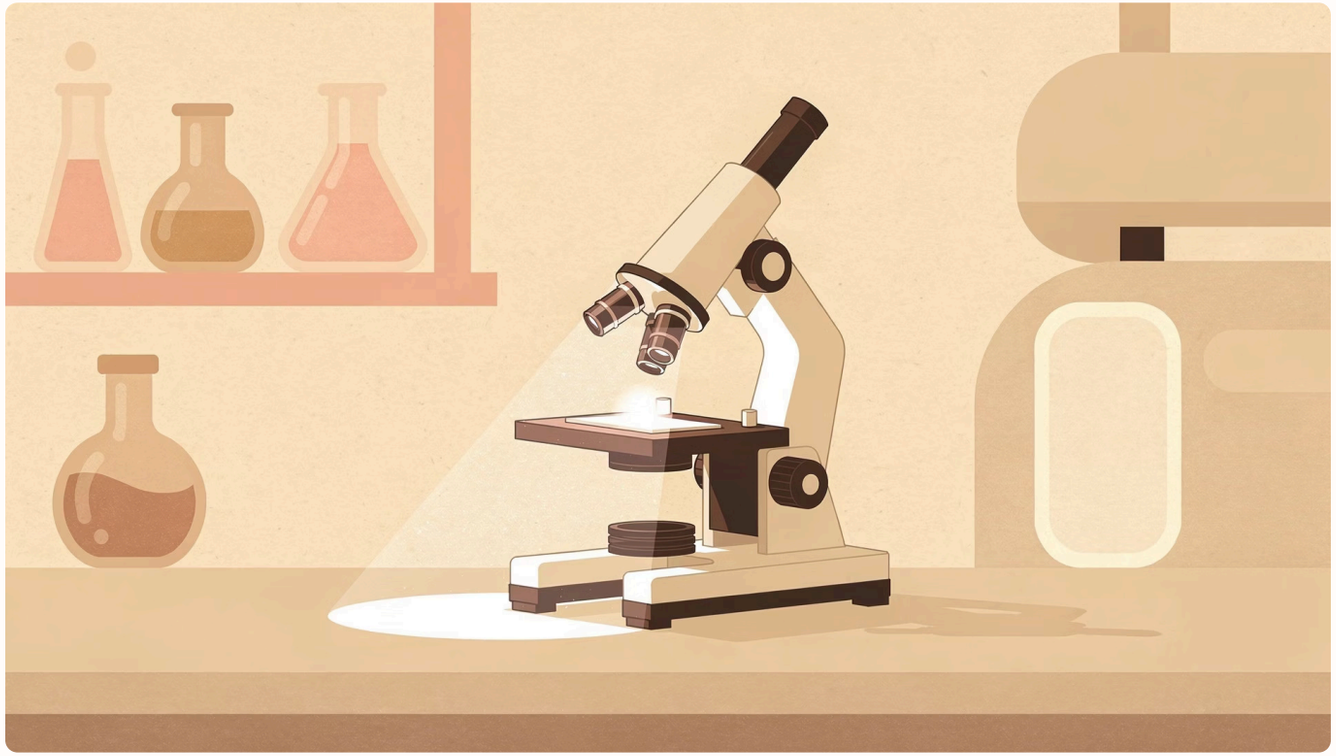
- Security and compliance requirements such as;
  - ARTS (Algorithmic Transparency Recording Standard - Mandator for MoJ)
  - MoJ Engineering AI Governance Framework
- Reputational stakes in Justice and wider Government
- We are at the mercy of our Azure Overlords (Stealth model updates, model deprecation, capacity issues, etc)

⚠ In our world, reputation is everything. We're not just handling sensitive data.. we're building trust for AI within the Justice system. This trust can be broken instantly if our systems are wrong

CHAPTER 2 KEY DISTINCTION

Model Evaluation ≠ System Evaluation

With commercial models, we don't control training data or model internals. But we **still own the behaviour** of the systems we deploy.



Model Evals (Vendor)

- General capability
- Benchmark performance
- Often opaque for commercial models

System Evals (Ours)

- Use-case fitness
- Prompts + workflow + guardrails
- Security posture in app context

Building your golden dataset

> **Golden Dataset** = fixed set of inputs + context + expected outcomes (verified by SMEs)

01

Start with production-ish examples

Pull from real world examples and redact all PII along with any other sensitive information

02

Capture edge cases deliberately

Balance the dataset to not overfit to happy path use cases. A good rule of thumb: **60%** common, **30%** edge cases, **10%** adversarial/abuse

03

Keep your golden dataset updated

As the system usage changes and new edge cases appear, update the dataset to reflect current reality

04

Build evaluation into your CI/CD pipelines

So quality checks happen automatically on every change

⊗ Avoid using synthetic data (AI generated) within your golden dataset where possible

Baseline First: Benchmark Two Models

Run your golden dataset against at least two models to establish a baseline for quality, safety, cost, and latency.

A sanity check against vibes

If Model A looks "good" but Model B is dramatically better on the same data, you may be overfitting prompts to one model's quirks

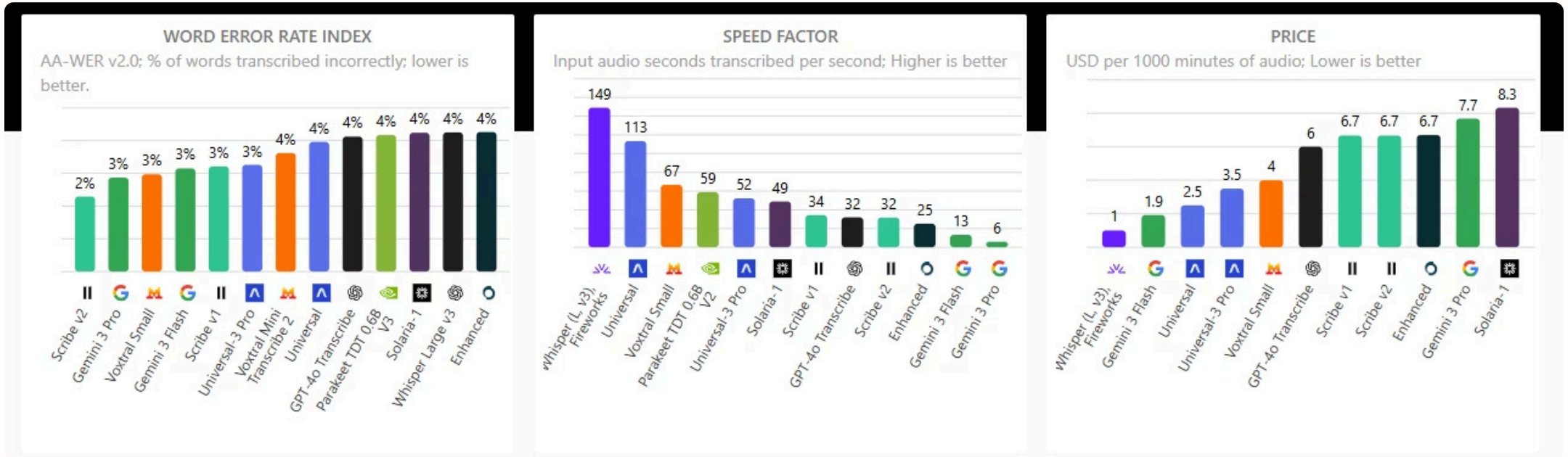
A floor and a ceiling

Even a rough two-model run gives you a **baseline floor** (cheaper option) and a **baseline ceiling** (best-case model), anchoring every future decision in evidence.

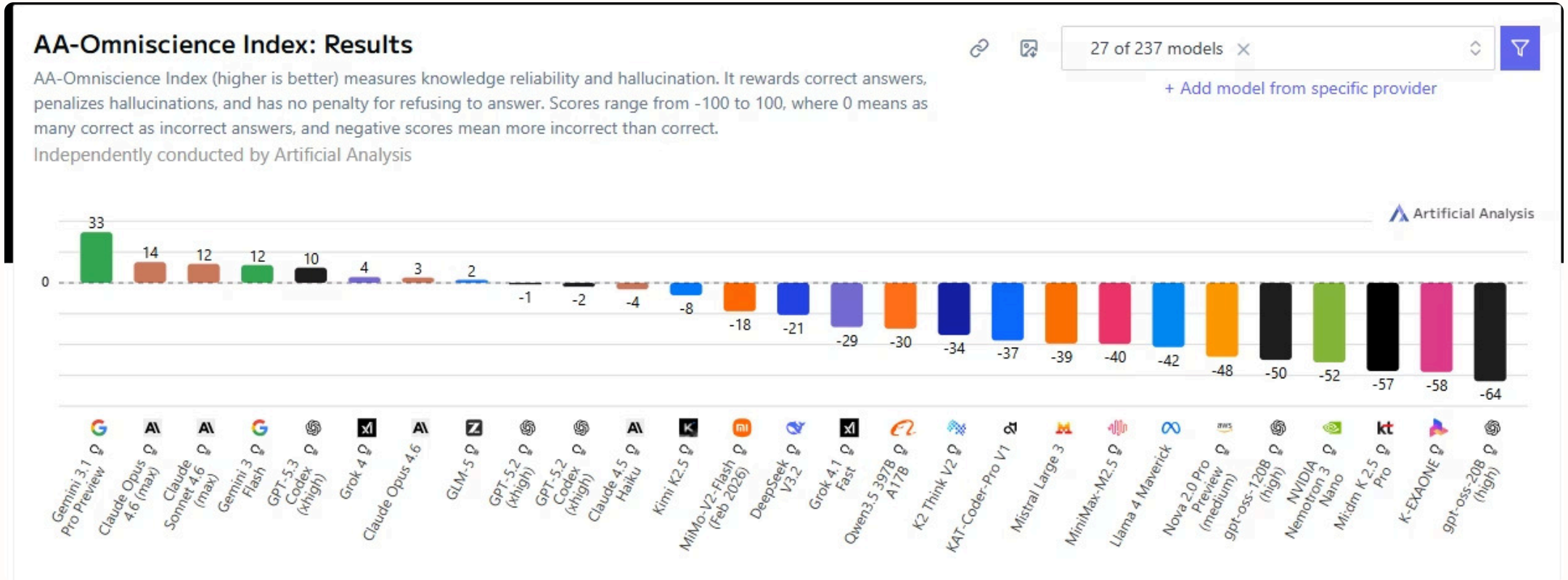
Artificial Analysis can help

To be used as a starting point for model selection, not as proof the model works for specific use cases.

Artificial Analysis - Speech-to-text evals



Artificial Analysis - Omniscience Index



CHAPTER 4 THE FRAMEWORK

Four Questions Every Engineer Should Know

1 **Does it work for the task?**

Useful and correct enough for the intended job, including weird edge cases

2 **Does it fail safely?**

Handles uncertainty, bad inputs, unsupported asks, and misuse appropriately

3 **Does it work reliably in context?**

Performs across real workflow conditions, integrations, and edge conditions

4 **Can we detect when it starts to fail?**

Observability, feedback loops, and review mechanisms are in place

CHAPTER 5 EDGE CASES

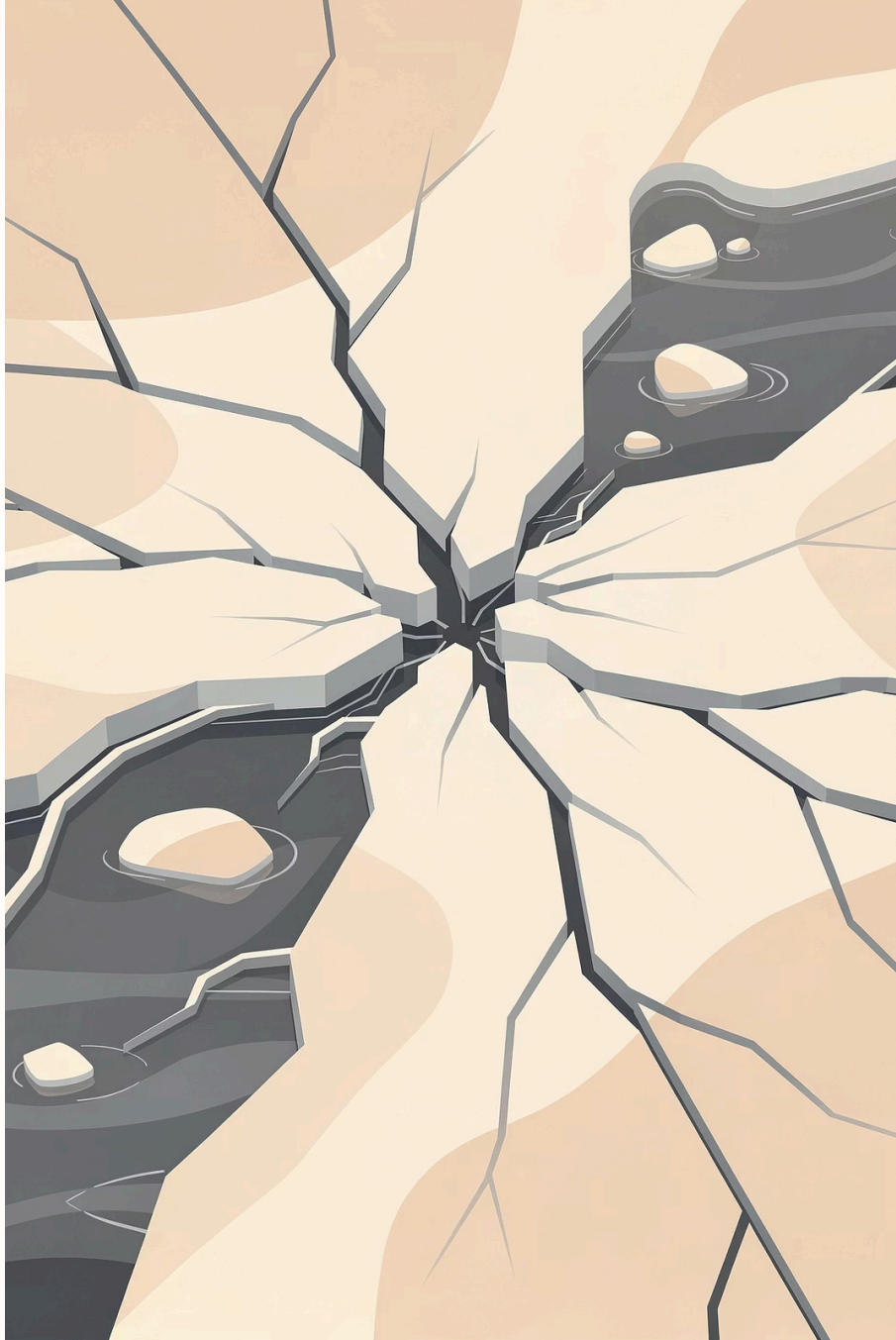
Edge Cases Are Where AI Systems Break

STT/Transcription

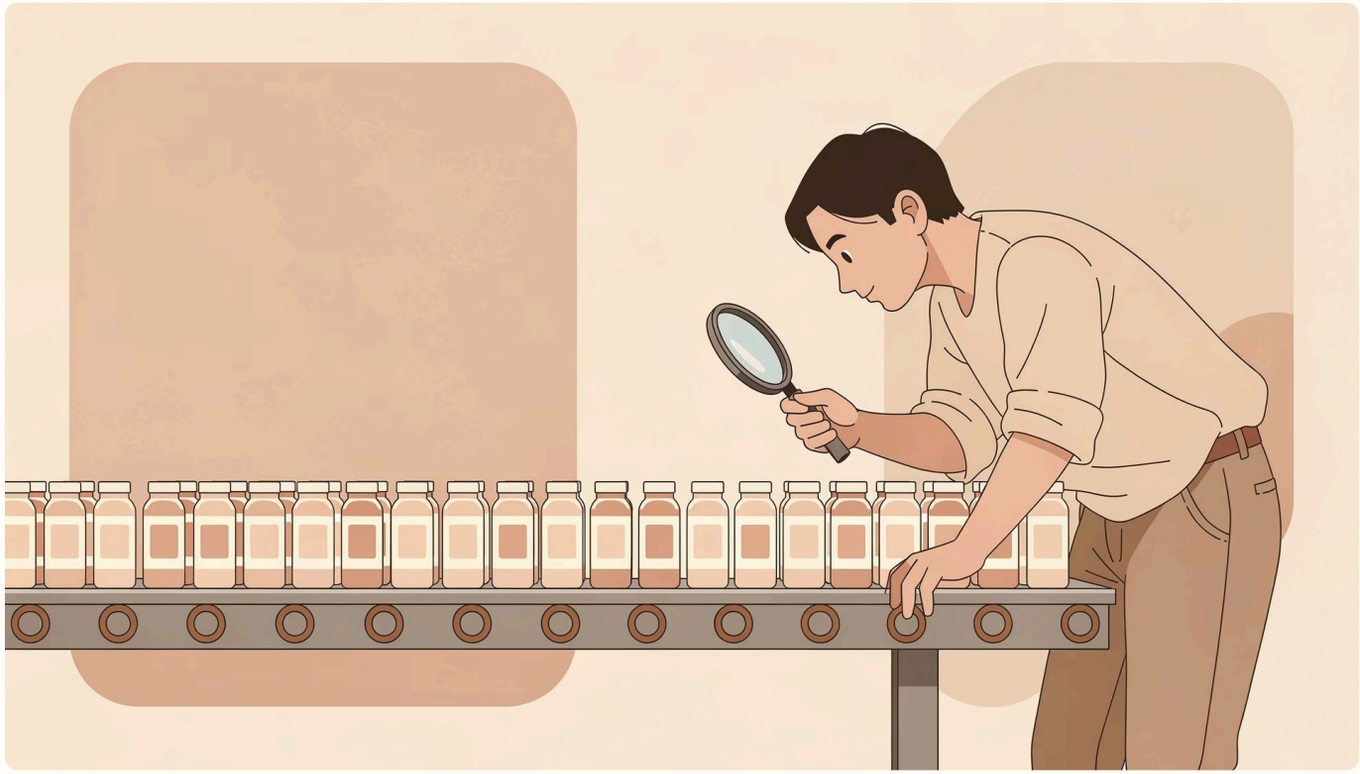
Accents, background noise, speaker overlap, domain acronyms, portions of recording without audio

LLM Generation

Conflicting instructions, request refusal , adversarial input, context window limits



Prove It's Fit for Purpose



If the system has multiple AI components / use cases - each one should be tested

- **Define the task**

  What is this system supposed to do?

- **Define good output**

  What does success look like concretely?

- **Define failure modes**

  What are the bad outputs and key security risks?

- **Test the integrated workflow**

  Not just isolated prompts - include refusal and abuse cases

Measure Behaviour, Not Just Uptime

Traditional Observability

- Uptime
- Latency
- Error Rate

AI Observability

- User feedback (good, bad, why)
- Corrections & retries
- Unsafe input patterns
- Unsafe output patterns
- Drift after changes (Prompt, Model update, etc)

Security Evaluation Is Use-Case-Specific

Different AI apps have different risks, but teams can use the same risk-thinking approach.

What can it access? Who can access it?

What can it generate?

What can it trigger?

What if it's wrong or abused?

Security posture is a property of the **application and workflow**, not just the underlying model.

Example Security Evaluation Activities

01

Threat model the workflow

Assets, actors, trust boundaries, high-impact failure modes

02

Adversarial testing / red teaming

Prompt injection, data exfiltration, jailbreaks, policy bypass

03

Tool and action misuse tests

"Make it do the wrong thing" scenarios and permission boundary tests

04

Abuse and escalation paths

Refusal quality, safe fallback behaviour, human handoff correctness

LLM Security Risks - OWASP Top 10 LLM

Risk	Description
LLM01: Prompt Injection	Manipulating LLMs via crafted inputs
LLM02: Insecure Output Handling	Neglecting to validate LLM outputs
LLM03: Training Data Poisoning	Tampered training data impairs LLM models
LLM04: Model Denial of Service	Overloading LLMs with resource-heavy operations
LLM05: Supply Chain Vulnerabilities	Depending on compromised components, services or datasets
LLM06: Sensitive Information Disclosure	Failure to protect against disclosure of sensitive info in outputs
LLM07: Insecure Plugin Design	LLM plugins processing untrusted inputs with insufficient access control
LLM08: Excessive Agency	Granting LLMs unchecked autonomy to take action
LLM09: Overreliance	Failing to critically assess LLM outputs
LLM10: Model Theft	Unauthorized access to proprietary large language models

LLM Security Mitigations - Azure Content Filters

Filter category	Status	Default setting	Applied to prompt or completion?	Description
Prompt Shields for direct attacks (jailbreak)	GA	On	User prompt	Filters / annotates user prompts that might present a Jailbreak Risk. For more information about annotations, visit Foundry content filtering.
Prompt Shields for indirect attacks	GA	Off	User prompt	Filter / annotate Indirect Attacks, also referred to as Indirect Prompt Attacks or Cross-Domain Prompt Injection Attacks, a potential vulnerability where third parties place malicious instructions inside of documents that the generative AI system can access and process. Requires: Document embedding and formatting.
Protected material - code	GA	On	Completion	Filters protected code or gets the example citation and license information in annotations for code snippets that match any public code sources, powered by GitHub Copilot. For more information about consuming annotations, see the Protected material concepts guide
Protected material - text	GA	On	Completion	Identifies and blocks known text content from being displayed in the model output (for example, song lyrics, recipes, and selected web content).
Groundedness	Preview	Off	Completion	Detects whether the text responses of large language models (LLMs) are grounded in the source materials provided by the users. Ungroundedness refers to instances where the LLMs produce information that is non-factual or inaccurate from what was present in the source materials. Requires: Document embedding and formatting.
Personally identifiable information (PII)	Preview	Off	Completion	Filters information that can be used to identify a particular individual, such as a name, address, phone number, email address, social security number, driver's license number, passport number, or similar information.

⚠ Don't rely on prompt based mitigations alone, they can pretty much always be bypassed

LLM Security Mitigations - Prompt hierarchy

System Prompt

Defines model behavior, constraints, and guardrails. Highest trust, never exposed to end users.

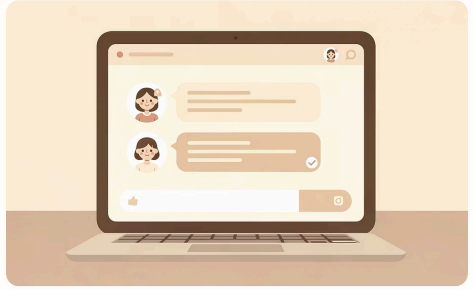
User Input

End-user input, treated as untrusted. Validated and scoped by system prompt.

Tool Call Results

Data from external tools. Lowest trust, potentially adversarial (injection risk).

Off-the-Shelf Models, Custom Evaluation



Chatbot / LLM Assistant

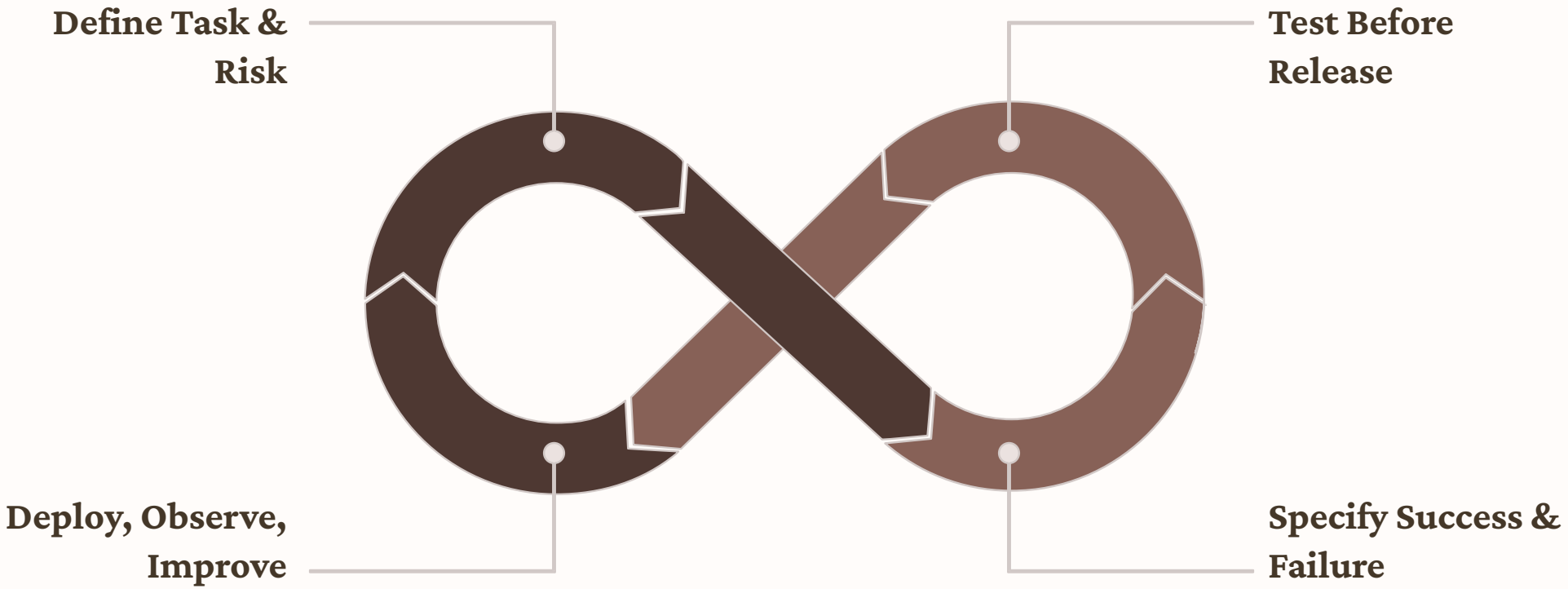
- Answer usefulness
- Groundedness / hallucination rate
- Refusal and escalation quality
- Retry rate / time-to-useful-answer

STT / Transcription / Translation

- Domain term accuracy
- WER (Word Error Rate)
- Named entity preservation
- Performance across accents/noise

What is an acceptable success rate for your use case? usually it's pretty high (95-99%)

A Simple Evaluation Loop



Start small. Be repeatable. Improve with evidence. The aim is **confidence proportional to risk**, not perfection.

Common Traps That Create Rework & Incidents

"The vendor tested it, so we're covered"

Vendor benchmarks don't test your workflow, your data, or your users.

Testing only happy paths

If edge cases are "phase 2", incidents are phase 1.5.

No explicit success criteria

Without a definition of "good", you can't measure improvement.

Shipping without logging

If you can't diagnose behaviour, you can't fix it.

CALL TO ACTION

Start Small, Build Confidence, Ship Faster

We should be aiming for a **repeatable way to build confidence** into AI delivery.

For every AI feature, answer five questions

What task? What does good look like? What are the risks? How will we test? What will we monitor?

We're Here to Help

Define metrics

We can help define evaluation metrics for your specific use case

Design tests

We can help design lightweight pre-release test suites

Set up observability

We can help identify the right production signals to monitor

The Endgame

> **"We earn trust by making reliability, performance and security measurable, repeatable, and visible - in real-world use, not just demos."**

Radical Transparency (Internal first, externally where appropriate)

we can clearly state and measure what the system is good at, where it struggles, and the conditions we tested under.

Real-world performance evidence

golden datasets + live telemetry show task success rates, error modes, etc across representative scenarios (including edge cases).

Security as a tested property

adversarial evals (prompt injection, data exfiltration, tool misuse) are baked into our CI/CD pipeline and we can demonstrate our controls working under attack.

Governance as a delivery enabler

guardrails and evidence packs that turn approvals into a fast, repeatable check, not a debate over governance.

Questions?